
SynthFuzz: Fuzzing MLIR Compilers with Custom Mutation Synthesis

Ben Limpanukorn



Jiyuan Wang



Hong Jin Kang



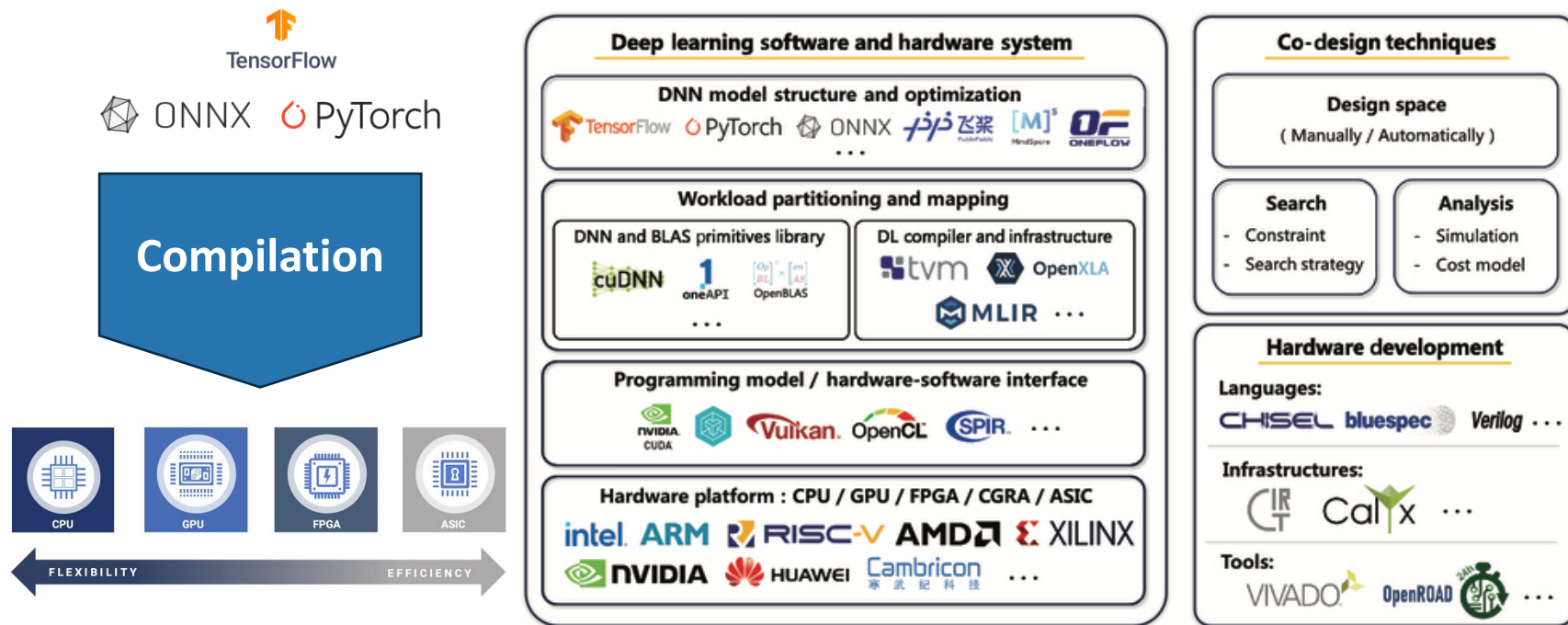
Eric Zitong Zhou



Miryung Kim



Context: ML Compilers & HW Accelerators

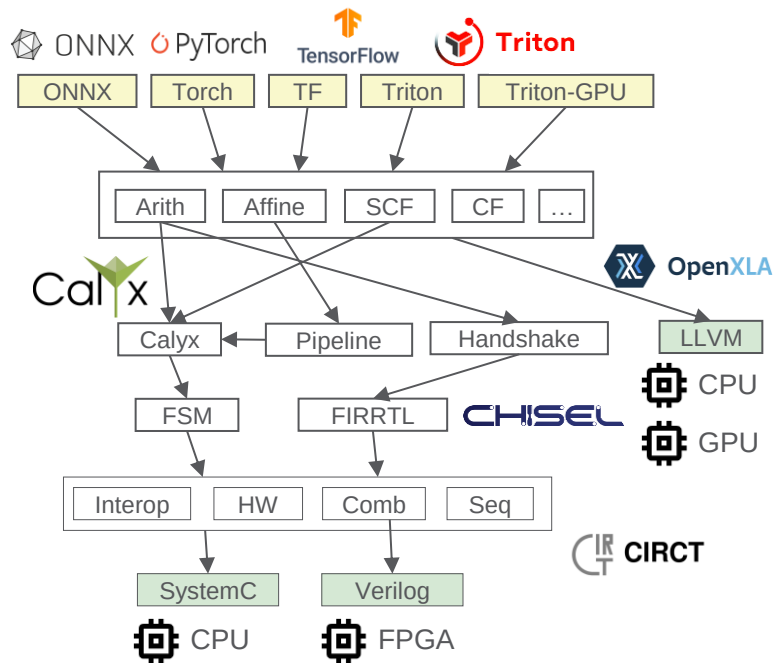


MLIR: An Extensible Compiler Framework

30+ projects have adopted MLIR since 2020

MLIR Compiler	# Compilation Passes	# Dialects	# Operations
LLVM (MLIR)	85	42	495
ONNX-MLIR	34	2	313
Triton	30	4	44
CIRCT	55	26	192
Total	204	74	1044

We need a testing strategy that can keep up with the pace of development.



How can we bootstrap dialect-specific custom generators
without significant manual effort?

Challenge 1: Dialect Specific Grammars are Needed

Grammar-based generators require **dialect-specific grammars and continual maintenance**

Input generated with
base MLIR grammar



```
"xklj.ZkdD"(%ixm) () → (abdklx)  
"o.xm"(%xlksdo) {kld = 1} () → ()
```



Example of a valid
HW dialect input

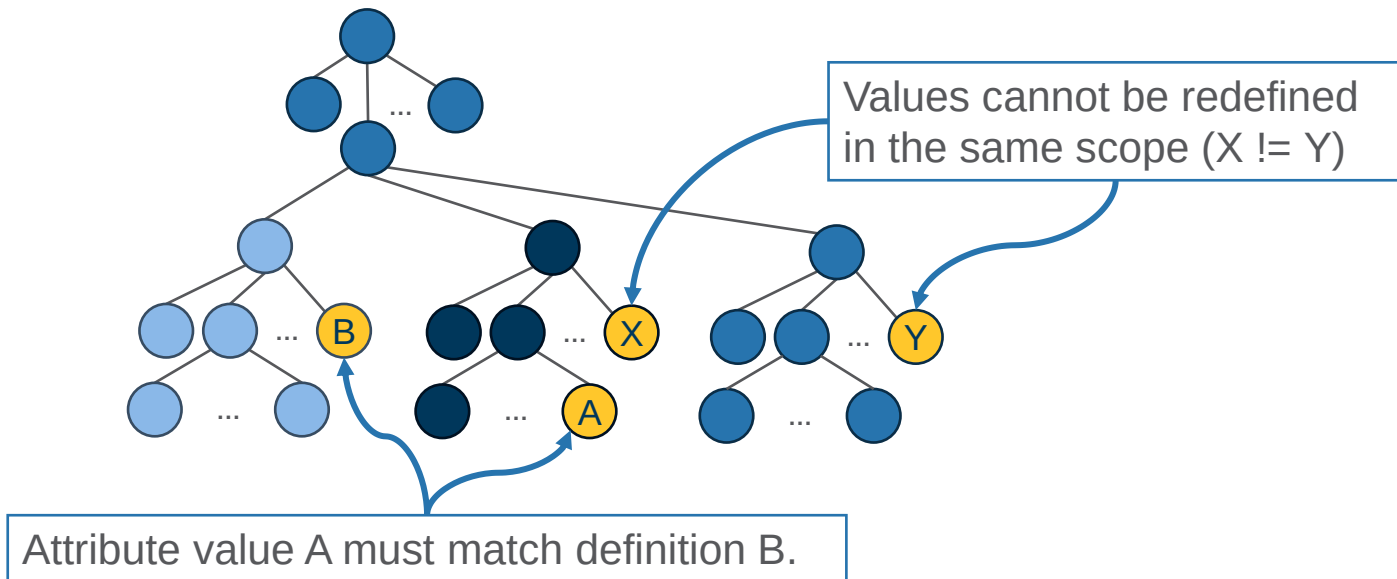
```
"hw.module"() ({  
  ^b(%a: i2):  
    "hw.output"(%a) → i2  
})
```



Generating with a base MLIR grammar yields test inputs that do not conform to each dialect specification.

Challenge 2: Context-sensitive constraints must be preserved.

Grammar-based fuzzing fails to satisfy **dialect-specific context-sensitive constraints** required for well-formed inputs

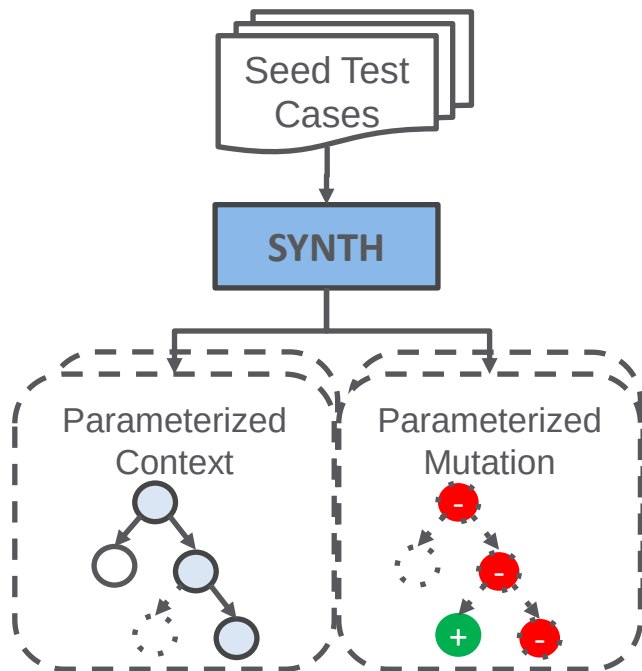


How can we bootstrap dialect-specific custom generators
without significant manual effort?



We synthesize custom mutations that **preserve**
dialect-specific constraints automatically.

Key Insight: Custom Mutation Synthesis



We use seed test cases to automatically synthesize custom mutations that encode semantic constraints.

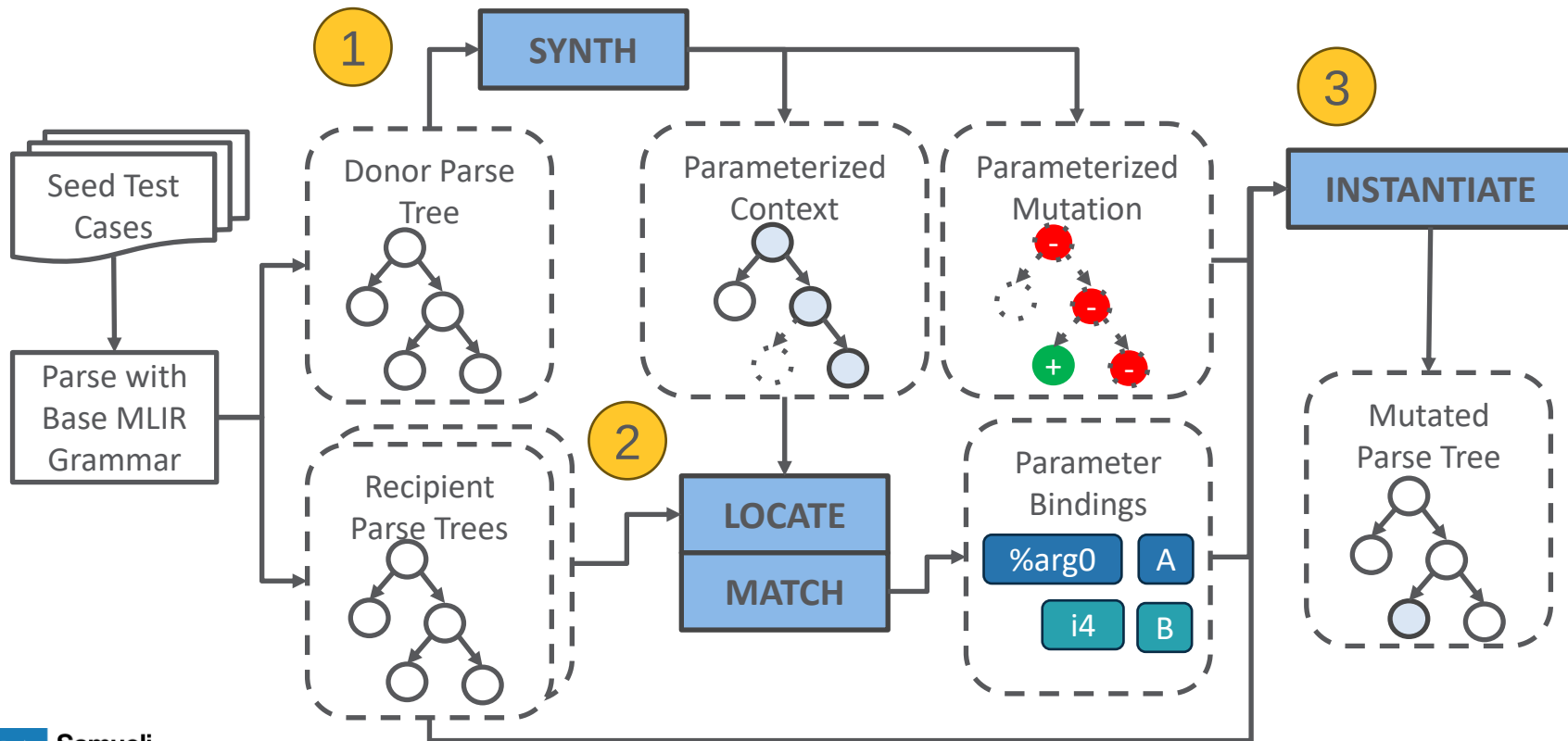
Positioning: Where should the mutation be applied?

Substitution: How to adapt the inserted content to its new context?

[1] *Learning Syntactic Program Transformations from Examples*, Rolim et al. ICSE 2017

[2] *Systematic Editing: Generating Program Transformations from an Example*, Meng, et al. PLDI 2011

SynthFuzz Overview



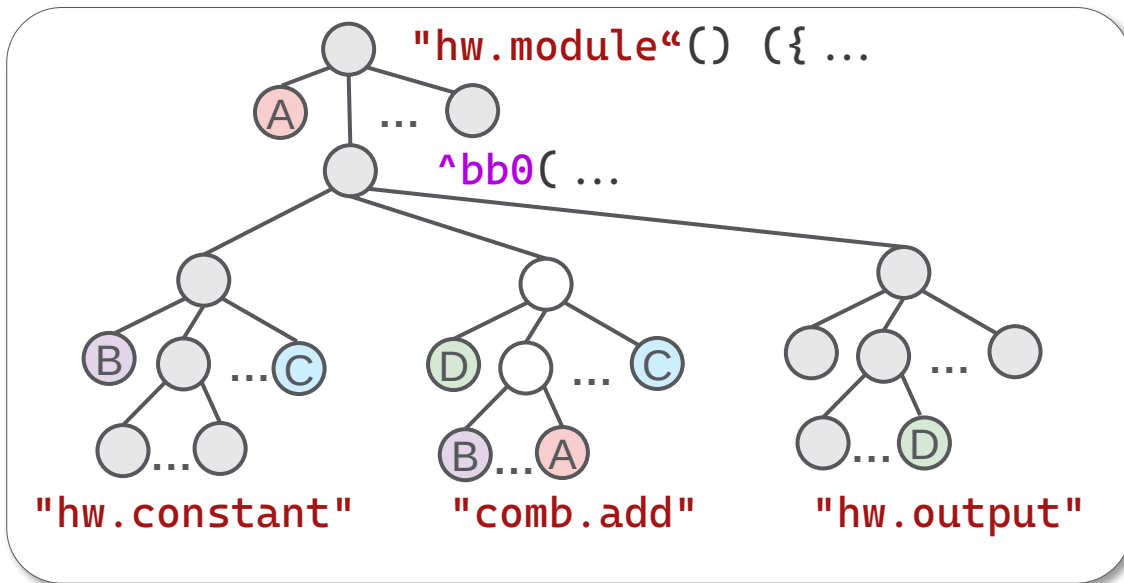
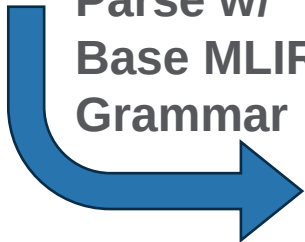
1 SYNTH: Creating Parameterized Mutation

Seed Test Case



MLIR

Parse w/
Base MLIR
Grammar



SynthFuzz constructs a mutation from the parse-tree of a seed test case by selecting a random sub-tree.

1 SYNTH: Creating Parameterized Mutation

Seed Test Case



MLIR

Parse w/
Base MLIR
Grammar

Context

"hw.module"() ({ ...

"%o1" appears both in the
"Mutation" and the "Context"

"hw.constant"

"comb.add"

"hw.output"

Mutation

Parameters

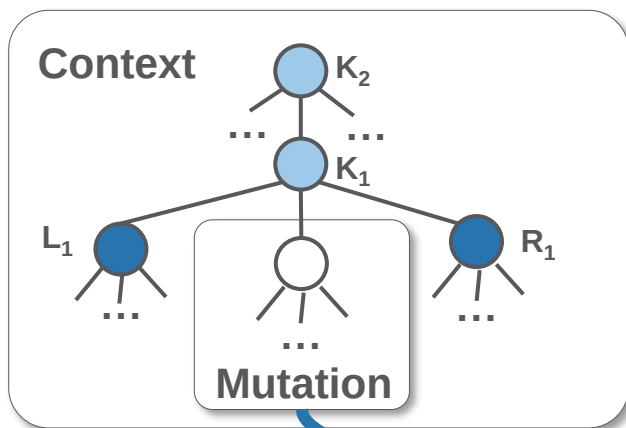
"%a1"	→	A
"%c1"	→	B
"i2"	→	C
"%o1"	→	D

SynthFuzz parameterizes the mutation content by inserting placeholders that will be concretized.

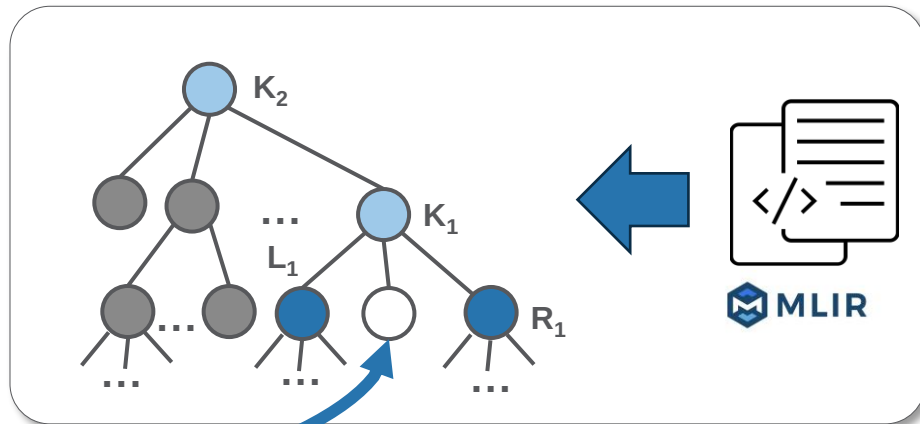
2

LOCATE: Positioning Mutation

Parameterized Mutation



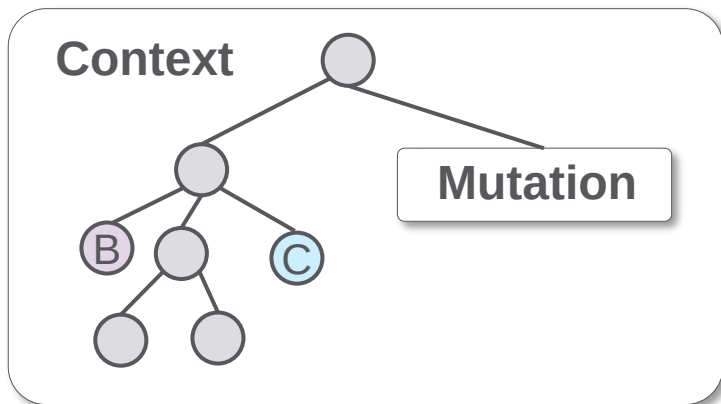
Recipient Test Case



SynthFuzz positions the mutation by matching the structure of the context to the parse tree of the recipient test case.

3 INSTANTIATE: Concretizing Mutation

Parameterized Mutation



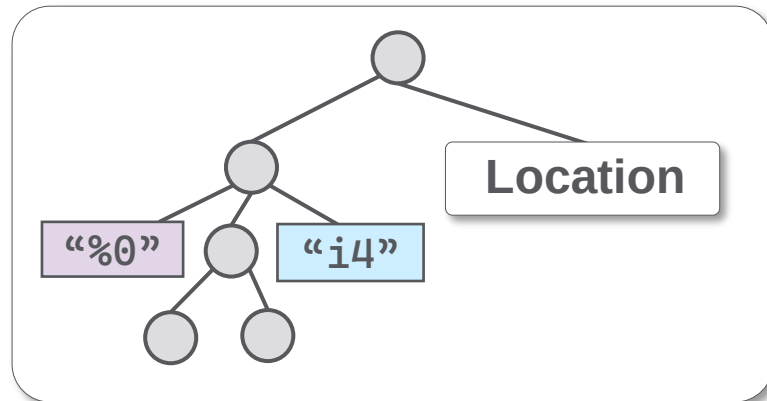
Parameter Bindings

B → %0

C → i4



Recipient Test Case



SynthFuzz concretizes the mutation by matching parameters to nodes in the recipient.

Research Questions

RQ1: How effective is SynthFuzz in terms of **increasing code coverage**?

RQ2: What is the **input space coverage** of SynthFuzz?

RQ3: Does **context-based positioning** of parametrized mutation improve the likelihood of a valid mutation?

RQ4: Does **parameterization** of the mutation content improve the likelihood of a valid mutation?

Input Space Coverage

Input Space Coverage measures the number of unique pairs operations related by **control and data dependencies** [1].

Data Dependency

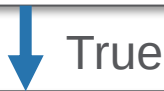
```
%c1 = "hw.constant"()
```



```
%o1 = "comb.add"(%a1, %c1)
```

Control Dependency

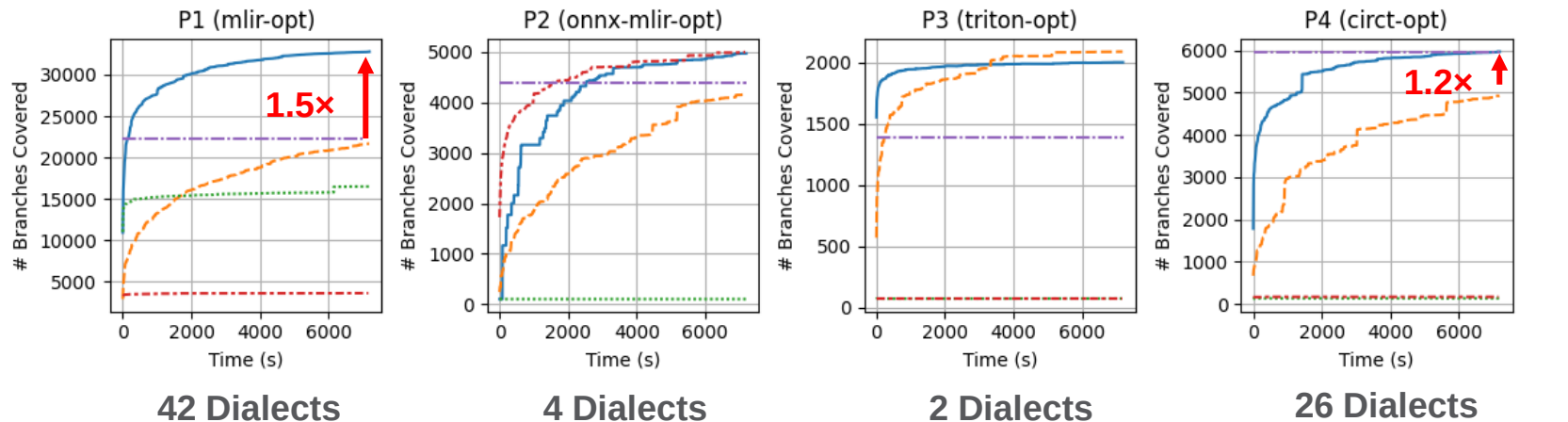
```
"affine.if"(%c)
```



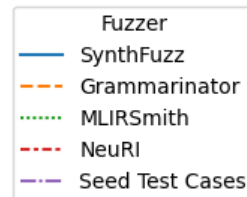
```
%o = "affine.add"(%b, %d)
```

[1] H. Wang *et al.*, "MLIRSmith: Random Program Generation for Fuzzing MLIR Compiler Infrastructure," 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), Luxembourg, Luxembourg, 2023

RQ1: Branch Coverage



SynthFuzz achieves the greatest improvement for compilers that use more dialects.



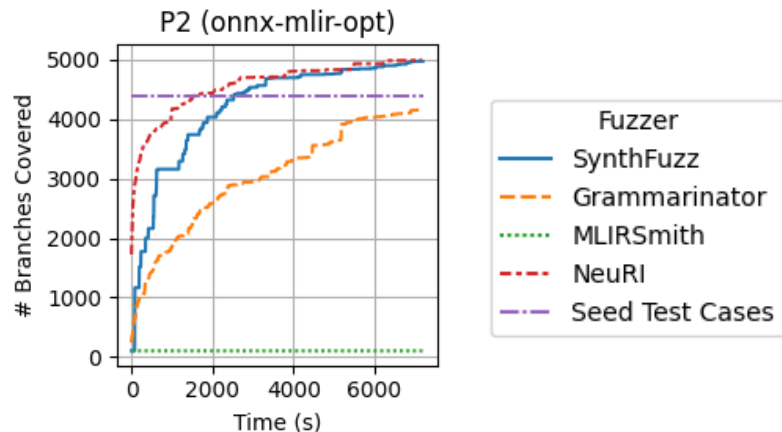
RQ2: Input Space Coverage

	MLIR-OPT		ONNX		Triton		CIRCT	
	Control	Data	Control	Data	Control	Data	Control	Data
SynthFuzz	129	100	23	24	15	4	86	40
Grammarinator	68	56	17	11	12	4	57	40
MLIRSmith	62	65	6	3	3	1	15	10
NeuRI	17	12	9	10	3	2	14	10

1.7× avg

Compared to custom MLIR generators, SynthFuzz can generalize to new dialects without hand-written mutators.

RQ1+RQ2: SynthFuzz v. Domain-Specific Generator



Input Space Coverage		
	Control	Data
SynthFuzz	23	24
Grammarinator	17	11
MLIRSmith	6	3
NeuRI	9	10

SynthFuzz can match NeuRI in terms of branch coverage and outperforms baselines in terms of input space coverage.

RQ3: Context-based Mutation Positioning

	Parameters	Branch Coverage	Input Space Coverage	Valid Tests
Ancestor	K=0	24764	100	677
	K=4	24987	90	749
Left	L=0	25055	94	684
	L=4	24713	98	729
Right	R=0	24657	97	706
	R=4	24958	98	731

Increasing ancestor-path, prefix, and postfix requirements for context positioning improves the proportion of valid test cases.

RQ4: Parameterized Mutation

	Violation Type	W/O Param	SynthFuzz (W/ Param)
Invalid	Dialect Specific	24%	38%
	General MLIR	30%	16%
	Invalid Options	39%	39%
Valid	N/A	7%	7%

Kernel must have a non-zero dimension

Value X is not defined.

Parameterizing mutations enables SynthFuzz to better exercise dialect-specific code.

Conclusion

- Testing MLIR-based compilers are challenging due to **rapidly evolving** dialects.
- To obviate writing *dialect-specific grammars and constraints*, we **synthesize custom mutations** from existing test cases.
- Fuzzing MLIR compilers with synthesized custom mutation **increases code and input space coverage**.
- Custom mutation synthesis can be applied to other domains to reduce effort of writing custom test generator.



Thank You

arxiv.org/abs/2404.16947

github.com/UCLA-SEAL/SynthFuzz



Ben Limpanukorn



Jiyuan Wang



Hong Jin Kang



Eric Zitong Zhou



Miryung Kim